

**LAPORAN PRAKTIKUM
PRAKTIKUM PEMROGRAMAN WEB 1
PERTEMUAN 10
JavaScript: Cara Penyisipan dan Output**



Disusun oleh:

Nama : Gloria Olive Vashti Indriani
NIM : 25/568243/SV/27404
Kelas : TRPL 2025 B2
Dosen Pengampu : Firma Syahrian, S.Kom., M.Cs.

**PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

PERTEMUAN 10

JavaScript

A. Tujuan Praktikum

1. Memahami perbedaan antara cara penyisipan JavaScript secara Embed, Inline, dan Eksternal ke dalam halaman HTML.
2. Mengetahui dan mempraktikkan berbagai metode output JavaScript, yaitu `console.log()`, `alert()`, `document.write()`, dan `innerHTML`.
3. Menerapkan logika percabangan (if-else) dan fungsi interaktif (`prompt`, `confirm`, `alert`) dalam pemrograman JavaScript.

B. Dasar Teori

JavaScript merupakan bahasa pemrograman berbasis skrip yang berjalan di sisi klien (client-side) dan digunakan untuk menambahkan interaktivitas pada halaman web. Menurut Flanagan [1], JavaScript adalah bahasa inti dari web modern yang memungkinkan pengembang untuk memanipulasi dokumen HTML secara dinamis, menangani peristiwa (event), dan berkomunikasi dengan server tanpa perlu me-refresh halaman.

Terdapat tiga cara utama dalam menyisipkan JavaScript ke dalam dokumen HTML. Pertama, Embed JavaScript yaitu kode ditulis langsung di dalam tag `<script>` yang ditempatkan di dalam dokumen HTML. Kedua, Inline JavaScript yaitu kode ditulis langsung pada atribut event sebuah elemen HTML seperti `onclick`. Ketiga, Eksternal JavaScript yaitu kode disimpan dalam file terpisah berekstensi `.js` dan dipanggil menggunakan atribut `src` pada tag `<script>` [2].

Dalam hal output, JavaScript menyediakan beberapa mekanisme untuk menampilkan data kepada pengguna. Metode `console.log()` digunakan untuk menampilkan informasi di konsol browser yang umumnya digunakan untuk keperluan debugging. Metode `alert()` menampilkan kotak dialog berupa pesan kepada pengguna. Metode `document.write()` menuliskan konten langsung ke dalam aliran dokumen HTML, sedangkan properti `innerHTML` digunakan untuk memanipulasi atau mengganti konten elemen HTML tertentu secara dinamis [3].

C. Pembahasan

1. File 1 JavaScript Embed, Inline, dan Eksternal

File pertama mendemonstrasikan tiga cara penyisipan JavaScript ke dalam halaman HTML sekaligus dalam satu dokumen. Berikut adalah kode lengkap file pertama:

```
<!DOCTYPE html>
<html>
<head>
  <title>Javascript Embed Inline Eksternal</title>

  <!-- Eksternal Javascript -->
```

```

    <script src="script.js"></script>
</head>

<body>

    <h2>Contoh Javascript</h2>

    <!-- Embed Javascript -->
    <script>
        document.write("<p> EMBED Javascript</p>");
        function pesanEksternal() {
            alert("Ini adalah EKSTERNAL Javascript");
        }
    </script>

    <br>

    <!-- Inline Javascript -->
    <button onclick="alert('Ini adalah INLINE Javascript')">
        Klik Inline
    </button>

    <br><br>

    <!-- Eksternal Javascript -->
    <button onclick="pesanEksternal()">
        Klik Eksternal
    </button>

</body>
</html>

```

Contoh Javascript

EMBED Javascript

Klik Inline

Klik Eksternal

127.0.0.1:54549 says

Ini adalah INLINE Javascript

OK

Contoh Javascript

EMBED Javascript

Klik Inline

Klik Eksternal

127.0.0.1:54549 says

Ini adalah EKSTERNAL Javascript

OK

a) Bagian <head> Cara Penyisipan Eksternal

Pada bagian <head>, terdapat tag <script src="script.js"></script>. Penggunaan atribut src menunjukkan cara menyisipkan JavaScript secara Eksternal, artinya kode JavaScript tidak ditulis langsung di dalam file HTML melainkan disimpan pada file terpisah bernama script.js. Metode ini merupakan pendekatan yang paling direkomendasikan dalam pengembangan web modern karena memisahkan logika dari struktur tampilan, sehingga kode lebih mudah dikelola, diuji, dan digunakan kembali di berbagai halaman.

b) Blok <script> di <body> Cara Penyisipan Embed

Di dalam tag <body>, terdapat blok <script> yang merupakan contoh JavaScript Embed, yaitu kode JavaScript ditulis langsung di dalam dokumen HTML namun tetap di dalam tag <script>. Pada blok ini terdapat dua hal: pertama, pemanggilan document.write("<p> EMBED Javascript</p>") yang langsung menuliskan sebuah paragraf HTML ke dalam dokumen pada saat halaman pertama kali dimuat oleh browser. Kedua, definisi fungsi pesanEksternal() yang apabila dipanggil akan menampilkan kotak dialog alert berisi teks "Ini adalah EKSTERNAL Javascript". Penempatan definisi fungsi di sini menunjukkan bahwa fungsi dalam blok embed maupun file eksternal berbagi ruang lingkup global yang sama.

c) Tombol Pertama Cara Penyisipan Inline

Elemen <button> pertama menggunakan atribut onclick="alert('Ini adalah INLINE Javascript')". Inilah yang disebut sebagai Inline JavaScript, yaitu kode JavaScript ditulis langsung di dalam atribut event sebuah elemen HTML. Ketika tombol diklik, browser langsung menjalankan ekspresi JavaScript yang tertulis di atribut onclick tersebut dan menampilkan kotak dialog pesan. Meskipun cara ini sederhana dan cepat ditulis, penggunaannya tidak disarankan untuk logika yang kompleks karena mencampur kode JavaScript dengan struktur HTML sehingga sulit dibaca dan dipelihara.

d) Tombol Kedua Memanggil Fungsi dari File Eksternal

Tombol kedua memiliki atribut onclick="pesanEksternal()" yang memanggil fungsi pesanEksternal(). Meskipun fungsi tersebut secara fisik didefinisikan di dalam blok embed pada file HTML yang sama, konsepnya menggambarkan bagaimana sebuah tombol HTML (Inline) dapat memanggil fungsi yang seharusnya berasal dari file eksternal. Apabila fungsi yang sama juga didefinisikan di dalam file script.js, maka fungsi yang terakhir dimuatlah yang akan digunakan. Bagian ini memperkuat pemahaman bahwa ketiga cara penyisipan dapat bekerja bersama-sama dalam satu dokumen HTML.

2. File 2 Macam Macam Output JavaScript

File kedua mendemonstrasikan empat metode output yang tersedia dalam JavaScript. Berikut adalah kode lengkap file kedua:

```
<!DOCTYPE html>
<html>
<head>
  <title>Output Javascript</title>
</head>

<body>

  <h2>Macam-Macam Output Javascript</h2>

  <button onclick="tampilConsole()">console.log()</button>
  <button onclick="tampilAlert()">alert()</button>
  <button onclick="tampilWrite()">document.write()</button>
  <button onclick="tampilInnerHTML()">innerHTML</button>

  <div id="hasil"></div>

</script>
```

```

// console.log()
function tampilConsole() {
    console.log("Ini output dari console.log()");
}

// alert()
function tampilAlert() {
    alert("Ini output dari alert()");
}

// document.write()
function tampilWrite() {
    document.write("Ini output dari document.write()");
}

// innerHTML
function tampilInnerHTML() {
    document.getElementById("hasil").innerHTML =
        "<h3>Ini output dari innerHTML</h3>";
}

</script>

</body>
</html>

```

Macam-Macam Output Javascript

Ini output dari innerHTML

127.0.0.1:54549 says

Ini output dari alert()

OK

Ini output dari document.write()

a) Struktur HTML Tombol dan Elemen Penampung

Halaman ini memiliki empat elemen `<button>` yang masing-masing memanggil satu fungsi JavaScript melalui atribut `onclick`. Di bawah tombol-tombol tersebut terdapat elemen `<div id="hasil">` yang berfungsi sebagai wadah kosong untuk menampilkan hasil output dari fungsi `tampilInnerHTML()`. Pemisahan antara tombol pemicu dan wadah penampil merupakan pola umum dalam pengembangan antarmuka web yang interaktif.

b) Fungsi `tampilConsole()`

Fungsi `tampilConsole()` dipanggil saat tombol `"console.log()"` diklik. Di dalamnya, perintah `console.log("Ini output dari console.log()")` mengirimkan pesan ke panel Konsol (Console) pada Developer Tools browser (dapat dibuka dengan menekan F12). Metode `console.log()` tidak menampilkan apapun di halaman web yang terlihat oleh pengguna biasa, sehingga sangat berguna bagi pengembang untuk melakukan proses debugging, memeriksa nilai variabel, atau mencatat alur eksekusi program tanpa mengganggu tampilan halaman.

c) Fungsi `tampilAlert()`

Fungsi tampilAlert() memanggil alert("Ini output dari alert()") yang akan memunculkan kotak dialog modal. Kotak dialog ini bersifat blocking, artinya seluruh interaksi pengguna dengan halaman web dihentikan sementara hingga pengguna menekan tombol "OK". Metode alert() sering digunakan untuk pemberitahuan penting yang harus diakui pengguna, namun dalam aplikasi web produksi modern penggunaannya umumnya diminimalisir karena dapat mengganggu pengalaman pengguna (user experience).

d) Fungsi tampilWrite()

Fungsi tampilWrite() menjalankan document.write("Ini output dari document.write()"). Perlu dipahami bahwa apabila document.write() dipanggil setelah halaman selesai dimuat (seperti pada kasus klik tombol ini), maka seluruh konten halaman yang sudah ada akan dihapus sepenuhnya dan digantikan hanya oleh teks yang baru ditulis. Perilaku destruktif ini membuat document.write() tidak lagi direkomendasikan untuk digunakan setelah halaman dimuat. Penggunaan yang tepat adalah hanya saat proses pemuatan halaman awal (initial page load).

e) Fungsi tampilInnerHTML()

Fungsi tampilInnerHTML() menggunakan document.getElementById("hasil") untuk mengakses elemen <div> yang memiliki id="hasil", kemudian mengubah properti innerHTML-nya menjadi string HTML "<h3>Ini output dari innerHTML</h3>". Berbeda dengan document.write() yang mengganti seluruh dokumen, innerHTML hanya memodifikasi konten dari elemen yang ditarget secara spesifik, sehingga elemen lain di halaman tidak terpengaruh. Inilah metode manipulasi DOM yang paling banyak digunakan karena aman, fleksibel, dan mendukung konten HTML dinamis.

3. File 3 Tugas JavaScript

File ketiga merupakan tugas yang mengintegrasikan berbagai konsep JavaScript yang telah dipelajari. Halaman ini menggunakan Bootstrap 5 untuk tampilan antarmuka yang lebih rapi dan profesional. Berikut adalah kode lengkap file ketiga:

```
<!DOCTYPE html>
<html>
<head>
  <title>Tugas Javascript</title>

  <!-- Bootstrap -->
  <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.
css">
</head>

<body class="container mt-4">

  <h2>Tugas Javascript</h2>

  <!-- Soal 1 -->
  <button class="btn btn-primary" onclick="banding()" ">
    Bandingkan Bilangan
  </button>

  <div id="hasil1" class="mt-3"></div>
<hr>
```

```

<!-- Soal 2 -->
<button class="btn btn-success" onclick="praktikan()">
  Cek Praktikan
</button>

<div id="hasil2" class="mt-3"></div>
<hr>

<!-- Soal 3 -->
<h3>Kalkulator Sederhana</h3>

<input type="number" id="a" placeholder="Angka 1">
<input type="number" id="b" placeholder="Angka 2">

<button class="btn btn-warning" onclick="hitung()">
  Hitung +
</button>

<div id="hasil3" class="mt-3"></div>

<script>

  // SOAL 1
  function banding() {

    let bil1 = prompt("Masukkan bilangan pertama");
    let bil2 = prompt("Masukkan bilangan kedua");

    if (bil1 > bil2) {
      hasil1.innerHTML =
        "Bilangan ke-1 lebih besar dari bilangan ke-2";
    }

    else if (bil1 < bil2) {
      hasil1.innerHTML =
        "Bilangan ke-1 lebih kecil dari bilangan ke-2";
    }

    else {
      hasil1.innerHTML =
        "Kedua bilangan sama";
    }
  }

  // SOAL 2
  function praktikan() {

    let jawab = confirm("Apakah anda praktikan PPW1?");

    if (jawab == true) {

      let nama = prompt("Masukkan nama");
      let nim = prompt("Masukkan NIM");
      let angkatan = prompt("Masukkan angkatan");

      hasil2.innerHTML = `
<table class="table table-bordered">
  <tr>
    <th>Nama</th>
    <td>${nama}</td>
  </tr>
  <tr>
    <th>NIM</th>
    <td>${nim}</td>
  </tr>
  <tr>
    <th>Angkatan</th>
    <td>${angkatan}</td>
  </tr>
</table>

```

```

    `;
  }

  else {
    hasil2.innerHTML =
      "Anda bukan praktikan PPW1, anda tidak boleh masuk!";
  }
}

// SOAL 3
function hitung() {

  let angka1 = parseInt(document.getElementById("a").value);
  let angka2 = parseInt(document.getElementById("b").value);

  let hasil = angka1 + angka2;

  hasil3.innerHTML = "Hasil = " + hasil;
}

</script>

</body>
</html>

```

Tugas Javascript

Bandingkan Bilangan

Bilangan ke-1 lebih kecil dari bilangan ke-2

Cek Praktikan

Nama	gloria olive
NIM	1234
Angkatan	2024

Kalkulator Sederhana

4 7 Hitung +

Hasil = 11

a) Struktur HTML Bootstrap dan Elemen Antarmuka

Halaman tugas menggunakan Bootstrap 5 yang diimpor melalui tag `<link>` yang mengarah ke CDN (Content Delivery Network) jsdelivr. Bootstrap menyediakan kelas-kelas CSS siap pakai seperti `btn btn-primary`, `btn btn-success`, `btn btn-warning`, `container`, dan `mt-3/mt-4` sehingga tampilan halaman menjadi lebih modern dan responsif tanpa perlu menulis CSS dari awal. Halaman ini terdiri dari tiga soal yang dipisahkan oleh tag `<hr>`, masing-masing memiliki tombol, div penampung hasil, dan pada soal ketiga terdapat dua input bertipe number.

b) Fungsi banding() Soal 1: Membandingkan Dua Bilangan

Fungsi `banding()` dipanggil saat tombol "Bandingkan Bilangan" diklik. Di dalam fungsi, dua variabel `bil1` dan `bil2` menerima nilai dari pengguna melalui dua pemanggilan `prompt()` secara berurutan. Fungsi `prompt()` menampilkan kotak dialog input yang meminta pengguna untuk mengetikkan teks, dan nilai yang diketikkan dikembalikan sebagai string. Selanjutnya, program menggunakan struktur percabangan `if-else if-else`

untuk membandingkan kedua nilai: jika $bil1 > bil2$ maka ditampilkan pesan bahwa bilangan pertama lebih besar, jika $bil1 < bil2$ maka ditampilkan pesan sebaliknya, dan jika keduanya sama maka ditampilkan pesan bahwa kedua bilangan sama. Hasil ditampilkan menggunakan `hasil1.innerHTML`, yang merupakan cara singkat untuk mengakses elemen dengan `id="hasil1"` dan memodifikasi kontennya.

c) Fungsi praktikan() Soal 2: Verifikasi Data Praktikan

Fungsi `praktikan()` diawali dengan pemanggilan `confirm("Apakah anda praktikan PPW1?")` yang menampilkan kotak dialog berisi pertanyaan dengan dua pilihan: tombol OK yang mengembalikan nilai `true` dan tombol Cancel yang mengembalikan nilai `false`. Nilai boolean tersebut disimpan dalam variabel `jawab`. Jika `jawab` bernilai `true` (pengguna menekan OK), program meminta tiga informasi tambahan secara berurutan melalui tiga `prompt()` yaitu nama, nim, dan angkatan. Ketiga data tersebut kemudian disusun menjadi tabel HTML menggunakan template literal JavaScript (sintaks backtick ` dengan ekspresi `${variabel}` di dalamnya) dan dimasukkan ke dalam `innerHTML` elemen `hasil2`. Penggunaan template literal memungkinkan penulisan string HTML multi-baris yang mudah dibaca dengan penyisipan variabel secara langsung. Jika pengguna menekan Cancel (`jawab` bernilai `false`), ditampilkan pesan penolakan akses.

d) Fungsi hitung() Soal 3: Kalkulator Penjumlahan

Fungsi `hitung()` mengimplementasikan kalkulator penjumlahan sederhana. Nilai dari dua elemen input dengan `id="a"` dan `id="b"` diambil menggunakan `document.getElementById("a").value` dan `document.getElementById("b").value`. Karena properti `.value` pada elemen input HTML selalu mengembalikan nilai bertipe string, maka diperlukan konversi menggunakan `parseInt()` agar nilai tersebut dapat digunakan dalam operasi aritmatika. Fungsi `parseInt()` mengubah string menjadi bilangan bulat (integer). Setelah kedua nilai dikonversi, operasi penjumlahan `angka1 + angka2` dilakukan dan hasilnya disimpan dalam variabel `hasil`. Nilai hasil kemudian ditampilkan pada elemen `hasil3` menggunakan `innerHTML` dengan format string `"Hasil = " + hasil`. Fungsi ini mengilustrasikan cara lengkap untuk mengambil input dari pengguna, memprosesnya, dan menampilkan hasilnya kembali ke halaman web.

<https://github.com/glovashti/tugas-pratikum-11.git>

D. Kesimpulan

Berdasarkan praktikum, dapat disimpulkan bahwa JavaScript dapat disisipkan ke dalam HTML melalui tiga cara, yaitu Embed (menggunakan tag `<script>` di dalam file HTML), Inline (langsung pada atribut event seperti `onclick`), dan Eksternal (melalui file `.js` terpisah yang dipanggil dengan atribut `src`). Di antara ketiganya, metode Eksternal paling direkomendasikan karena mendukung pemisahan kode yang lebih baik antara logika dan tampilan. Selain itu, JavaScript menyediakan beberapa mekanisme output yang memiliki fungsi dan kegunaan masing-masing, antara lain `console.log()` untuk debugging, `alert()` untuk menampilkan kotak dialog, `document.write()` untuk menulis langsung ke dokumen (tidak disarankan setelah halaman dimuat), serta `innerHTML` untuk memanipulasi konten

elemen HTML secara dinamis dan lebih aman. Melalui tugas praktikum, mahasiswa juga telah menerapkan percabangan if-else, penggunaan fungsi interaktif seperti prompt() dan confirm(), konversi tipe data dengan parseInt(), pengambilan nilai dari elemen input, serta pembuatan konten HTML dinamis menggunakan template literal JavaScript. Praktikum ini secara keseluruhan menekankan pentingnya integrasi JavaScript yang baik serta teknik dasar untuk menciptakan halaman web yang interaktif

E. Daftar Pustaka

- [1] D. Flanagan, JavaScript: The Definitive Guide, 7th ed. Sebastopol, CA: O'Reilly Media, 2020.
- [2] N. C. Zakas, Professional JavaScript for Web Developers, 4th ed. Indianapolis, IN: John Wiley & Sons, 2019.
- [3] MDN Web Docs, "JavaScript Guide," Mozilla Developer Network, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>. [Accessed: May 17, 2026].